Online Social Bookstore Application

Sprint Report

<ELEFTHERIOS TRAMPAS>

1 Introduction

1.1 Purpose

This project refers to a Social Bookstore Application in which users can trade their books. This means that they can upload different kind of books and request from others.

2 Use Cases

2.1 <Use Case 1>

Use case ID	Register and login
Actors	• **User**: The person who uses the application to register, browse books, and request a book. • **System**: The book store application.
Pre conditions	• The book store application is running. • The database has predefined books, authors, and categories.
Main flow of events	USER REGISTRATION: • The user navigates to the login page. • The user enters their username and password. • The system verifies the username and password using `UserDetailsService`. • If the credentials are correct, the user is logged in. USER LOGIN: • The user navigates to the login page. • The user enters their username and password. • The system verifies the username and password using `UserDetailsService`. • If the credentials are correct, the user is logged in
Alternative flow 1	[Alternative flow of events IF ANY that takes place in case of an exception]

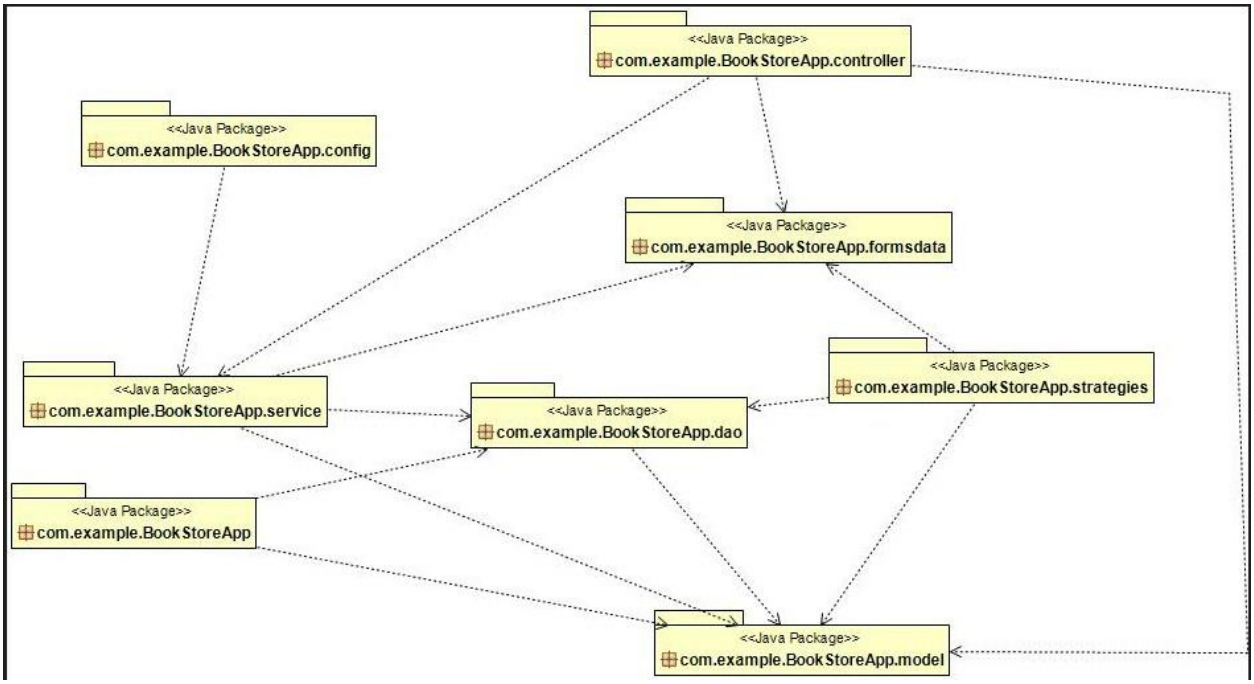
Alternative flow 2	.....
Post conditions	• The user is successfully registered in the system. • The user can browse books available in the store. • The user can request a book, which will be recorded in the system.

Use case ID	Saving Book Authors to User Profile
Actors	• **User**: The person who uses the application to create or update their user profile with favorite book authors. • **System**: The book store application.
Pre conditions	• The book store application is running. • The user is logged into their account and has navigated to the profile update page.
Main flow of events	**NAVIGATING TO PROFILE UPDATE**: 1. The user navigates to the profile update page. 2. The user fills in the form with their favorite book authors (comma-separated). **SUBMITING PROFILE UPDATE**: 1. The user submits the form. 2. The system extracts the author names from the form data. 3. The system iterates through the list of author names. 4. For each author name: 1. The system checks if the author already exists in the database using `bookAuthorService`. 2. If the author does not exist, the system creates a new `BookAuthor` entity and saves it using `bookAuthorService`. 3. If the author exists, the system retrieves the existing `BookAuthor` entity.

	4. The system adds the `BookAuthor` entity (new or existing) to the user profile's list of favorite authors. 5. The system saves the updated user profile to the database using `userProfileDAO`.
Alternative flow 1	
Alternative flow 2	
Post conditions	• The user profile is updated with the specified favorite book authors. • New authors not already in the database are created and saved. • Existing authors are linked to the user profile.

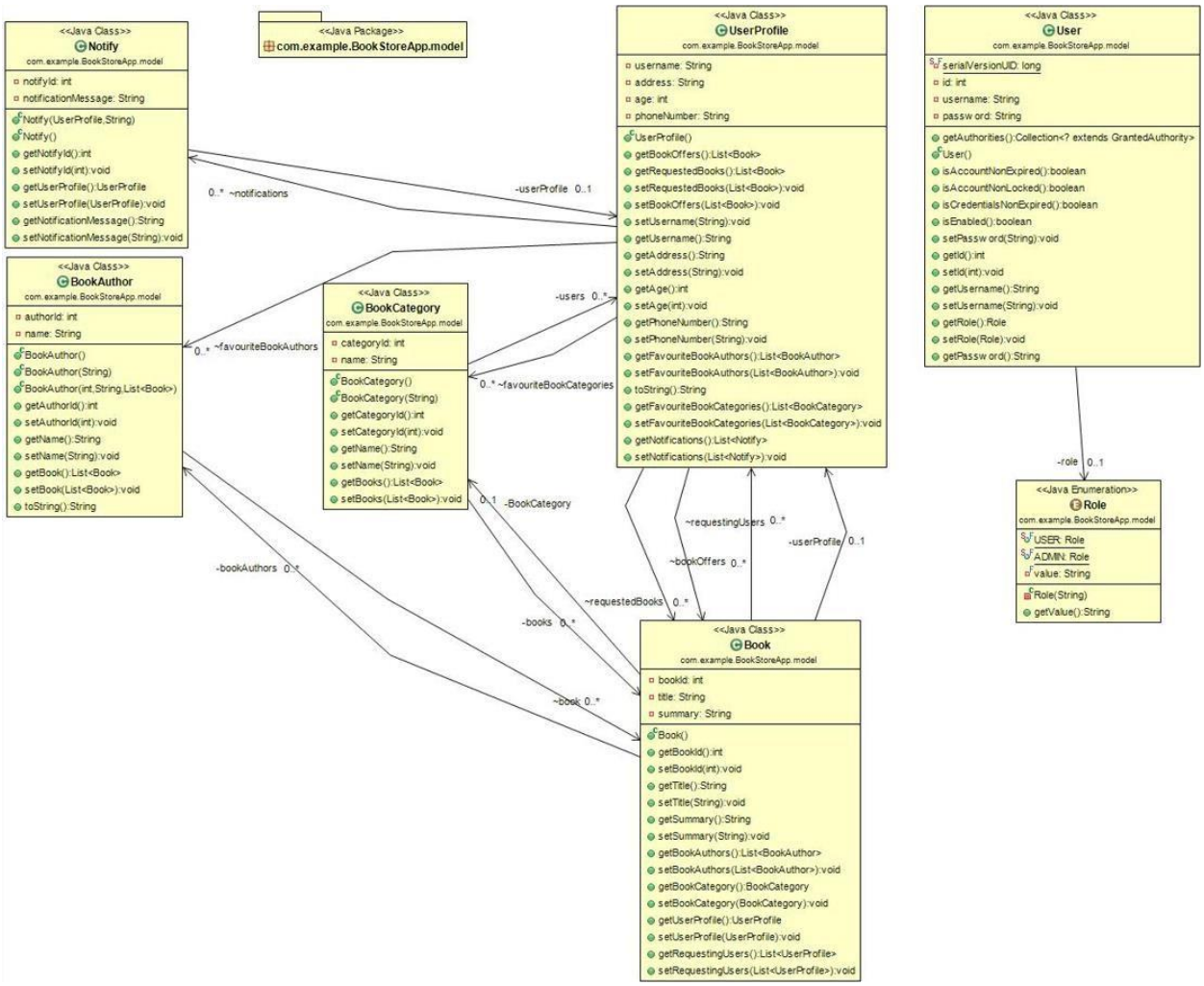
3 Design

3.1 Architecture

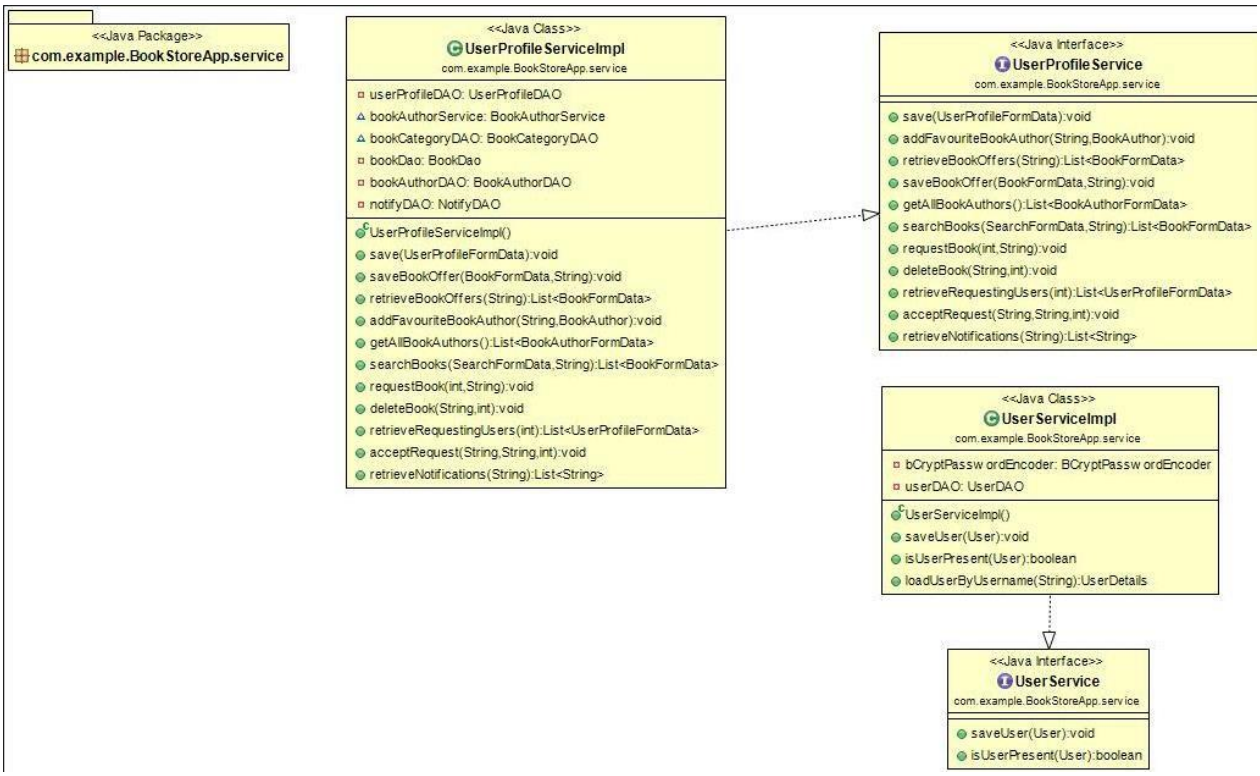


3.2 Design

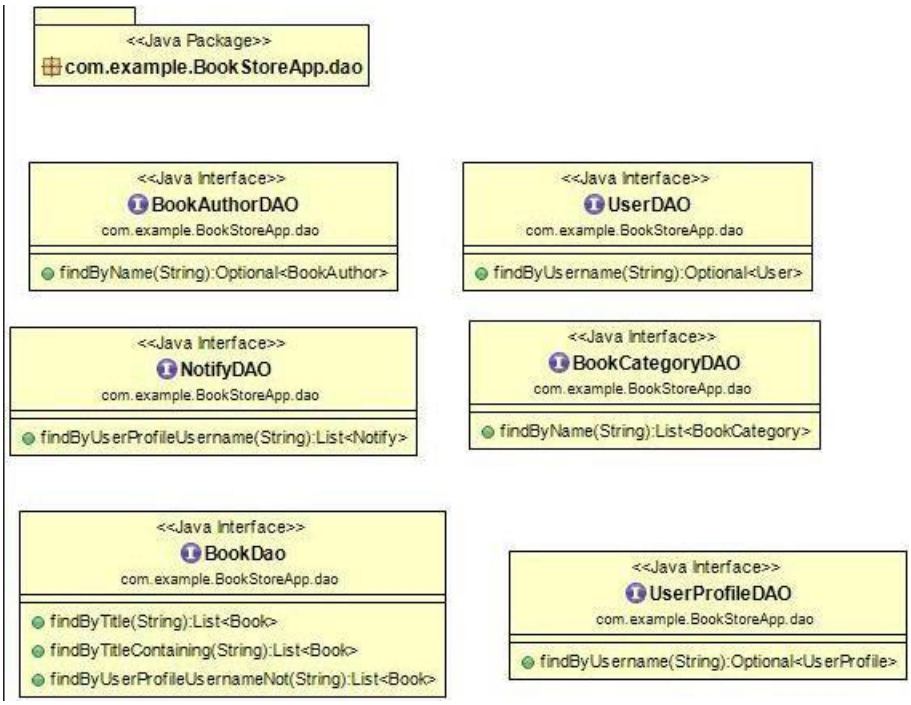
MODEL :


<<Java Class>>
Notify
com.example.BookStoreApp.model
notifyId: int
notificationMessage: String
Notify(UserProfile,String)
Notify()
getNotifyId():int
setNotifyId(int):void
getUserProfile():UserProfile
setUserProfile(UserProfile):void
getNotificationMessage():String
setNotificationMessage(String):void
<<Java Package>>
com.example.BookStoreApp.model
<<Java Class>>
UserProfile
com.example.BookStoreApp.model
username: String
address: String
age: int
phoneNumber: String
UserProfile()
getBookOffers():List<Book>
getRequestedBooks():List<Book>
setRequestedBooks(List<Book>):void
setBookOffers(List<Book>):void
setUsername(String):void
getUsername():String
getAddress():String
setAddress(String):void
getAge():int
setAge(int):void
getPhoneNumber():String
setPhoneNumber(String):void
getFavouriteBookAuthors():List<BookAuthor>
setFavouriteBookAuthors(List<BookAuthor>):void
toString():String
getFavouriteBookCategories():List<BookCategory>
setFavouriteBookCategories(List<BookCategory>):void
getNotifications():List<Notify>
setNotifications(List<Notify>):void
<<Java Class>>
User
com.example.BookStoreApp.model
serialVersionUID: long
id: int
username: String
password: String
getAuthorities():Collection<? extends GrantedAuthority>
User()
isAccountNonExpired():boolean
isAccountNonLocked():boolean
isCredentialsNonExpired():boolean
isEnabled():boolean
setPassword(String):void
getId():int
setId(int):void
getUsername():String
setUsername(String):void
getRole():Role
setRole(Role):void
getPassword():String
<<Java Class>>
BookAuthor
com.example.BookStoreApp.model
authorId: int
name: String
BookAuthor()
BookAuthor(String)
BookAuthor(int,String,List<Book>)
getAuthorId():int
setAuthorId(int):void
getName():String
setName(String):void
getBook():List<Book>
setBook(List<Book>):void
toString():String
<<Java Class>>
BookCategory
com.example.BookStoreApp.model
categoryId: int
name: String
BookCategory()
BookCategory(String)
getCategoryId():int
setCategoryId(int):void
getName():String
setName(String):void
getBooks():List<Book>
setBooks(List<Book>):void
<<Java Class>>
Book
com.example.BookStoreApp.model
bookId: int
title: String
summary: String
Book()
getBookId():int
setBookId(int):void
getTitle():String
setTitle(String):void
getSummary():String
setSummary(String):void
getBookAuthors():List<BookAuthor>
setBookAuthors(List<BookAuthor>):void
getBookCategory():BookCategory
setBookCategory(BookCategory):void
getUserProfile():UserProfile
setUserProfile(UserProfile):void
getRequestingUsers():List<UserProfile>
setRequestingUsers(List<UserProfile>):void
<<Java Enumeration>>
Role
com.example.BookStoreApp.model
USER: Role
ADMIN: Role
value: String
Role(String)
getValue():String
0..* ~notifications
-userProfile 0..1
0..* ~favouriteBookAuthors
-users 0..*
0..* ~favouriteBookCategories
0..1 -BookCategory
-bookAuthors 0..*
~requestingUsers 0..*
-userProfile 0..1
~bookOffers 0..*
~requestedBooks 0..*
-books 0..*
~book 0..*
-role 0..1

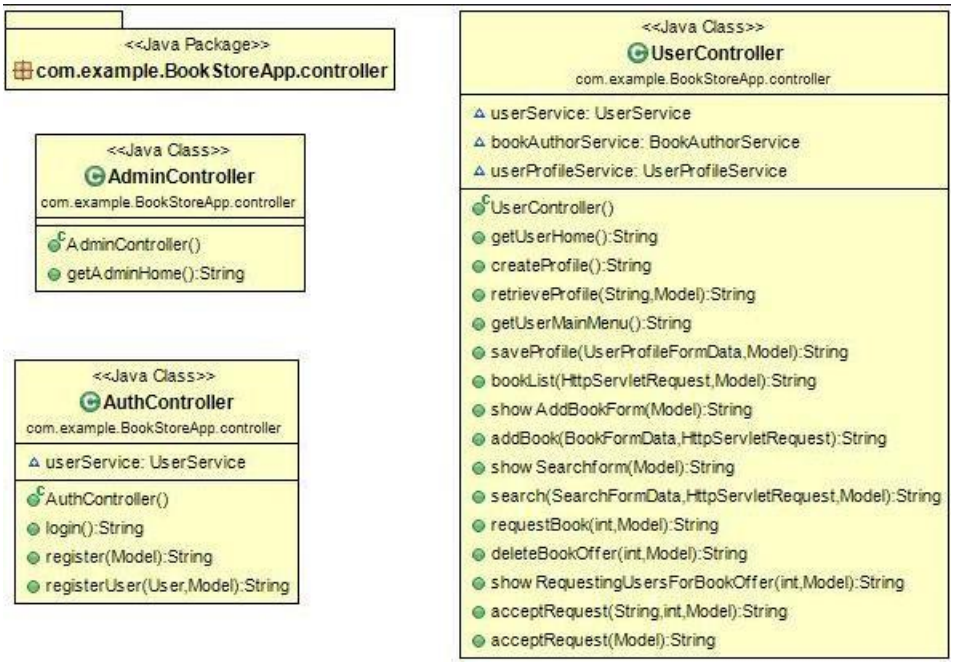
SERVICE :



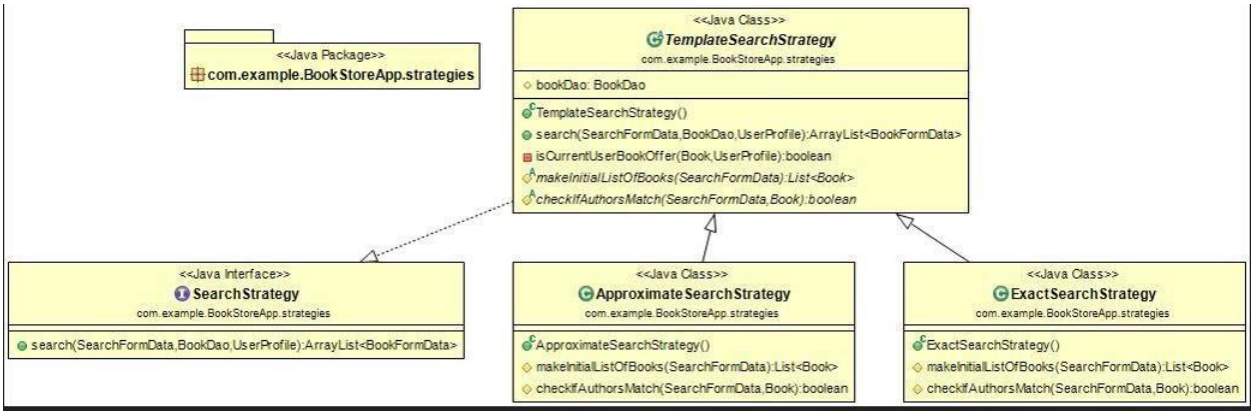
DAO – REPOSITORY :



CONTROLLERS :



STRATEGIES :



FORMS DATA :

